Deep Learning

Complete Revision Notes

AI/ML Engineering Revision Journey — Module 5 (Day 1, 2 & 3)

Part 1: ANN Fundamentals — Neurons, Forward/Backprop, Optimizers, Regularization
Part 2: CNN — Convolution, Pooling, Architectures, Computer Vision
Part 3: Sequential & Generative Models — RNN, LSTM, Attention, Transformers, GANs

Table of Contents

PART 1 — Deep Learning Fundamentals (ANN)
1. What is a Neural Network? & The Perceptron
2. Network Architecture & Activation Functions
3. Forward Propagation & Loss Functions
4. Backpropagation & Gradient Descent Variants
5. Optimizers (SGD, Momentum, RMSProp, Adam)
6. Weight Initialization & Vanishing/Exploding Gradients
7. Preventing Overfitting & Learning Rate Scheduling
PART 2 — Convolutional Neural Networks (CNN)
8. Why CNN? & The Convolution Operation
9. Stride, Padding & Pooling
10. Feature Maps, Flatten & Fully Connected Layers
11. Image Preprocessing & Data Augmentation
12. Transfer Learning
13. Popular CNN Architectures
14. Computer Vision Applications
PART 3 — Sequential & Advanced Deep Learning
15. RNN & The Vanishing Gradient Problem
16. LSTM & GRU
17. Bidirectional RNN & Sequence-to-Sequence
18. Attention & Transformers
19. Generative Models (Autoencoders, VAE, GANs)
20. Framework Revision & Model Evaluation
Interview Revision & End-to-End Project Recap
Key Takeaways

PART 1

Deep Learning Fundamentals — Artificial Neural Networks

1. What is a Neural Network? & The Perceptron

A **Neural Network** is layers of neurons, each computing **activation(W·x + b)**, stacked so the network can learn increasingly complex functions from data.

The **Perceptron** is the original, simplest neural network — a single neuron that learns a straight-line decision boundary. Update rule for each misclassified point:

```
w = w + lr * (y_true - y_pred) * x
b = b + lr * (y_true - y_pred)
```

2. Network Architecture & Activation Functions

Layer	Role
Input Layer	Receives raw features (no computation)
Hidden Layer(s)	Learn intermediate representations
Output Layer	Produces the final prediction

Activation	Range	Typical Use
Sigmoid	(0, 1)	Binary classification output
Tanh	(-1, 1)	Hidden layers (zero-centered)
ReLU	[0, ∞)	Default for hidden layers
Leaky ReLU	(-∞, ∞)	Fixes ReLU's dying-neuron problem
Softmax	(0, 1), sums to 1	Multi-class classification output

3. Forward Propagation & Loss Functions

Forward Propagation passes data through the network layer by layer: **Z = A_prev · W + b**, then **A = activation(Z)**.

Loss	Formula	Used for
MSE	mean((y - y_hat)^2)	Regression
Binary Cross-Entropy	-mean(y*log(y_hat) + (1-y)*log(1-y_hat))	Binary classification
Categorical Cross-Entropy	-mean(sum(y*log(y_hat)))	Multi-class classification

4. Backpropagation & Gradient Descent Variants

Backpropagation uses the chain rule to compute how much each weight contributed to the error, working backward from the output. **Gradient Descent** then updates weights: **w = w - learning_rate · dL/dw**.

Variant	Batch Size	Characteristics
Batch GD	Entire dataset	Stable but slow; one update per epoch
Stochastic GD (SGD)	1 sample	Fast, noisy; can escape local minima
Mini-Batch GD	16-256 typical	Best of both — the standard in practice

Epoch = one full pass through the training data. **Batch Size** = samples per weight update. **Iteration** = one weight update. **iterations per epoch = ceil(samples / batch_size)**.

5. Optimizers

Optimizer	Idea
SGD	Plain gradient descent step
Momentum	Moving average of past gradients accelerates consistent directions
RMSProp	Adaptive per-parameter LR via moving average of squared gradients
Adam	Momentum + RMSProp combined — the most widely used default

```
# Adam update rule
m = beta1*m + (1-beta1)*grad          # 1st moment (momentum)
v = beta2*v + (1-beta2)*grad**2       # 2nd moment (RMSProp-style)
m_hat = m / (1 - beta1**t)            # bias correction
v_hat = v / (1 - beta2**t)
w -= lr * m_hat / (sqrt(v_hat) + eps)
```

6. Weight Initialization & Vanishing/Exploding Gradients

Init Strategy	Variance	Best paired with
Zero init	0	Never — breaks symmetry, network can't learn
Xavier / Glorot	1 / fan_in	Sigmoid, Tanh
He	2 / fan_in	ReLU, Leaky ReLU

Vanishing Gradients — in deep networks with saturating activations (Sigmoid/Tanh), gradients shrink as they multiply backward through layers, so early layers barely learn. **Exploding Gradients** — the opposite; gradients grow unstably (mitigated with gradient clipping). ReLU + He initialization is the standard fix for vanishing gradients in deep nets.

7. Preventing Overfitting & Learning Rate Scheduling

Technique	How it works
Dropout	Randomly disables a fraction of neurons each training step
L1 / L2 Regularization	Penalizes large weights in the loss function
Early Stopping	Stops training once validation loss stops improving
Batch Normalization	Normalizes each layer's inputs (zero mean, unit variance)

Learning Rate controls step size — too high overshoots, too low is slow to converge. **LR Scheduling** (step decay, exponential decay) starts high for fast progress, then decays for fine-grained convergence.

```
# Step decay                                    # Exponential decay
lr = initial_lr * (drop ** floor(epoch/epochs_drop))   lr = initial_lr * exp(-k * epoch)
```

PART 2

Convolutional Neural Networks (CNN)

8. Why CNN? & The Convolution Operation

Fully connected networks flatten images (destroying spatial structure) and need huge parameter counts. **CNNs** scan small **filters/kernels** across the image, sharing weights across all positions — preserving spatial structure with far fewer parameters, and learning a hierarchy of features (edges → textures → parts → objects).

```
output[i,j] = sum( image[i:i+k, j:j+k] * kernel )   # slide the kernel across the image
```

9. Stride, Padding & Pooling

Term	Meaning
Stride	How many pixels the filter moves each step
Padding	Zero-border added so filters can cover edge pixels / control output size
Max Pooling	Keeps the strongest activation in each region
Average Pooling	Keeps the average activation in each region

```
output_size = (input_size - kernel_size + 2*padding) / stride + 1
```

10. Feature Maps, Flatten & Fully Connected Layers

A conv layer applies **many filters** in parallel, each producing its own **feature map**. After several Conv+Pool stages, feature maps are **flattened** into a 1D vector and fed into **Fully Connected (Dense)** layers for the final prediction:

```
Image -> [Conv -> ReLU -> Pool] x N -> Flatten -> Dense -> Dense -> Output
```

11. Image Preprocessing & Data Augmentation

- **Preprocessing:** Resizing to a fixed shape, Normalization (scale pixels to [0,1]), Grayscale conversion, Cropping
- **Data Augmentation:** Flips, Rotations, Brightness/Contrast changes, Zoom, Noise injection — artificially expands training data and reduces overfitting

12. Transfer Learning

Approach	What's trained	When to use
Feature Extraction	Only new final layer(s); pretrained base frozen	Small dataset, similar domain
Fine-Tuning	New layers + some/all pretrained layers (low LR)	Larger dataset, or different domain

```
base_model = ResNet50(weights="imagenet", include_top=False, input_shape=(224,224,3))
base_model.trainable = False        # Feature Extraction

for layer in base_model.layers[-20:]:
    layer.trainable = True          # Fine-Tuning: unfreeze top layers, train at low LR
```

13. Popular CNN Architectures

Architecture	Year	Key Idea
LeNet-5	1998	The original CNN — Conv+Pool stacks for digit recognition
AlexNet	2012	Deeper + ReLU + Dropout + GPU training — sparked the DL boom
VGG	2014	Very deep, uniform 3x3 conv filters stacked repeatedly
GoogLeNet (Inception)	2014	Parallel filter sizes per layer via "Inception modules"
ResNet	2015	Skip/residual connections enable very deep networks (50-150+ layers)
DenseNet	2017	Every layer connects to every other layer — max feature reuse
EfficientNet	2019	Scales depth, width & resolution together for best efficiency

14. Computer Vision Applications

Task	What it does
Image Classification	Assigns a single label to the entire image
Object Detection (YOLO)	Locates + classifies multiple objects via bounding boxes, in one forward pass
Semantic Segmentation	Classifies every pixel into a category, producing a pixel-level mask

PART 3

Sequential Models & Advanced Deep Learning

15. RNN & The Vanishing Gradient Problem

RNNs process a sequence step by step, maintaining a hidden state that carries information forward, using the **same weights at every timestep** (weight sharing across time):

```
h_t = tanh(x_t @ Wxh + h_(t-1) @ Whh + bh)
y   = h_T @ Why + by
```

Trained via **Backpropagation Through Time (BPTT)**. Just like deep ANNs, repeated multiplication through many timesteps causes **vanishing gradients**, making plain RNNs weak at learning long-range dependencies — this motivated LSTM and GRU.

16. LSTM & GRU

Gate (LSTM)	Formula	Role
Forget Gate	f_t = sigma(x_t*Wf + h_(t-1)*Uf + bf)	What to discard from cell state
Input Gate	i_t = sigma(x_t*Wi + h_(t-1)*Ui + bi)	What new info to add
Output Gate	o_t = sigma(x_t*Wo + h_(t-1)*Uo + bo)	What to expose as output

```
C_t = f_t * C_(t-1) + i_t * tanh(x_t@Wc + h_(t-1)@Uc + bc)   # cell state update
h_t = o_t * tanh(C_t)                                          # new hidden state
```

The forget gate lets gradients flow across many timesteps largely unchanged — solving the vanishing gradient problem. **GRU** simplifies this to two gates (Update, Reset) with no separate cell state — fewer parameters, often comparable performance.

17. Bidirectional RNN & Sequence-to-Sequence

Bidirectional RNN runs a forward and a backward RNN and combines both hidden states, so every timestep sees both past AND future context.

Sequence-to-Sequence (Encoder-Decoder) handles input/output sequences of different lengths (translation, summarization):

```
Input Sequence -> [Encoder RNN/LSTM] -> Context Vector -> [Decoder RNN/LSTM] -> Output Sequence
```

18. Attention & Transformers

Attention removes the fixed-context-vector bottleneck — the decoder can look back at ALL encoder states and learn which matter most for each output token.

```
Attention(Q, K, V) = softmax(Q @ K.T / sqrt(d_k)) @ V
```

Self-Attention applies this within a single sequence (Q, K, V from the same input) — directly relating any two positions regardless of distance. **Multi-Head Attention** runs several attention heads in parallel, each learning different relationships.

```
Input Embeddings + Positional Encoding
   -> [Multi-Head Self-Attention -> Add & Norm -> Feed-Forward -> Add & Norm] x N   (Encoder)
   -> [Masked Self-Attention -> Cross-Attention -> Feed-Forward] x N                (Decoder)
   -> Output Probabilities
```

Transformers have no recurrence, so they process sequences in parallel (much faster to train than RNNs) — this architecture underlies BERT, GPT, T5, and virtually all modern LLMs.

19. Generative Models

Autoencoders

Compress data into a small latent representation (Encoder), then reconstruct the original input from it (Decoder) — trained to reproduce its own input:

```
Input -> Encoder -> Latent (bottleneck) -> Decoder -> Reconstructed Output
```

Variational Autoencoders (VAE)

Makes the encoder output a probability distribution (mean μ, variance σ²) instead of a single point, sampled via the **reparameterization trick**:

```
z = mu + sigma * epsilon          where epsilon ~ N(0, 1)
Loss = ||x - decoder(z)||^2  +  KL( N(mu, sigma^2) || N(0, 1) )
```

The KL Divergence term gives the latent space a smooth, continuous structure — enabling generation of new samples by decoding random latent points.

GANs — Generative Adversarial Networks

Two networks compete: a **Generator** tries to create realistic fake data; a **Discriminator** tries to catch the fakes. Trained as a minimax game:

```
min_G max_D   E[log D(x)]  +  E[log(1 - D(G(z)))]
```

Notoriously tricky to train (mode collapse, instability) — part of why diffusion models have become popular alternatives for image generation.

20. Framework Revision & Model Evaluation

Keras Workflow

```
model = models.Sequential([
    layers.Dense(64, activation="relu", input_shape=(20,)),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train, y_train, validation_split=0.2, epochs=100,
          callbacks=[EarlyStopping(patience=5), TensorBoard(log_dir="./logs")])
model.save("model.keras")
```

PyTorch Workflow

```
model = Net()
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
for epoch in range(100):
    optimizer.zero_grad()
    loss = criterion(model(X_train), y_train)
    loss.backward()
    optimizer.step()
torch.save(model.state_dict(), "model.pth")
```

Model Evaluation Metrics (same for ANN, CNN, RNN, Transformer)

Metric	What it measures
Accuracy	Overall proportion of correct predictions
Precision	Of predicted positives, how many were correct
Recall	Of actual positives, how many were caught
F1 Score	Harmonic mean of Precision and Recall
ROC-AUC	Ability to separate classes across all thresholds
Confusion Matrix	Full breakdown of TP / TN / FP / FN

Interview Revision — Common Questions

Q: Why do we need activation functions?

Without non-linear activations, any number of stacked layers collapses into one linear layer.

Q: Why is ReLU preferred over Sigmoid in hidden layers?

Sigmoid saturates and causes vanishing gradients in deep nets; ReLU's gradient is 0 or 1 and is cheaper to compute.

Q: What causes overfitting and how do you prevent it?

The model memorizes training data/noise; prevented via Dropout, L1/L2, Early Stopping, more data, or a simpler model.

Q: Parameter vs Hyperparameter?

Parameters (weights/biases) are learned; hyperparameters (LR, layers, batch size) are set before training.

Q: Why do CNNs use fewer parameters than fully connected nets on images?

Weight sharing — the same small filter slides across the whole image.

Q: Why do LSTMs solve the vanishing gradient problem RNNs have?

The cell state acts as a near-linear memory highway gated by learned forget/input gates.

Q: What problem does Attention solve in Seq2Seq?

Removes the fixed-context-vector bottleneck by letting the decoder access all encoder states directly.

Q: Why can Transformers train faster than RNNs?

They process the whole sequence in parallel; RNNs must process timesteps sequentially.

Q: GAN vs VAE?

VAE optimizes reconstruction + KL-divergence (stable, blurrier); GAN optimizes an adversarial minimax game (sharper, less stable).

Q: How do you choose the number of layers/neurons?

Start simple, use validation performance and hyperparameter search rather than guessing.

End-to-End Deep Learning Project Recap

```
1. Problem Definition        -> What are we predicting? Success metric?
2. Data Collection            -> Gather raw data
3. Data Preprocessing         -> Clean, normalize, encode, augment
4. Train/Val/Test Split       -> Prevent leakage, enable honest evaluation
5. Model Architecture Choice  -> ANN / CNN / RNN / Transformer, based on data type
6. Training                   -> Forward -> Loss -> Backprop -> Optimizer step
7. Regularization              -> Dropout, L2, Early Stopping, Augmentation
8. Hyperparameter Tuning      -> LR, batch size, architecture size
9. Evaluation                 -> Accuracy/F1/AUC or MAE/RMSE/R2
10. Error Analysis             -> Inspect misclassified/high-error examples
11. Deployment                 -> Export model, serve via API, monitor
12. Monitoring & Retraining    -> Track drift, retrain on new data
```

Key Takeaways

Part 1 — ANN Fundamentals

- A neural network learns activation(W·x + b) through stacked layers, trained via forward propagation + backpropagation + gradient descent.
- Adam is the go-to optimizer; He initialization pairs with ReLU, Xavier with Sigmoid/Tanh.
- Dropout, L1/L2, Early Stopping, and Batch Normalization each fight overfitting differently.

Part 2 — CNN

- Convolution + weight sharing lets CNNs learn spatial patterns with far fewer parameters than fully connected nets.
- Pooling shrinks feature maps; Flatten + Dense layers turn spatial features into a final prediction.
- Transfer Learning (feature extraction / fine-tuning) leverages pretrained models like ResNet for new tasks.

Part 3 — Sequential & Generative Models

- RNNs process sequences with a shared hidden state; LSTM/GRU gates solve the vanishing gradient problem for long sequences.
- Attention lets models weigh relevant context directly; Transformers (self-attention, no recurrence) underlie modern LLMs.
- Autoencoders, VAEs, and GANs are the core generative model families, each with different tradeoffs between stability and output quality.

Module 5 complete: Day 1 (ANN Fundamentals) + Day 2 (CNN) + Day 3 (Sequential & Generative Models) — a full Deep Learning revision. Next: NLP, advanced Computer Vision, LLMs, and RAG.